

1. **P:Jakie klasy zostały zaprojektowane i jakie odpowiedzialności pełni każda z nich?**

O: W projekcie zaimplementowano klasę bazową Pracownik, która przechowuje dane wspólne(imię,nazwisko,ID), definiuje ona również interfejs, który jest potem wykorzystywany jako interfejs polimorficzny.

Klasy pochodne:

- PracownikEtatowy(pensja stała)
- PracownikGodzinowy(wyliczanie pensji na podstawie stawki I godzin)
- PracownikProwizyjny(obliczanie wynagrodzenia jako procent od sprzedaży)

2. **P:Jakie cechy są wspólne dla wszystkich pracowników, a jakie różnią klasy pochodne?**

O:

Wspólne:

- ☐ imię
- ☐ nazwisko
- ☐ identyfikator[id]
- ☐ funkcjonalność obliczania pensja I wyświetlania danych

Różnice:

- ☐ Parametry wejściowe do obliczeń dla różnych typów pracowników
- ☐ Logika matematyczna w metodzie **obliczPensje()**

3. **P:Które pola w projekcie zostały ukryte jako private i dlaczego?**

O: Wszystkie pola przechowujące stan(imię,nazwisko etc.) są prywatne.

Cel takiego działania polega na ograniczyć modyfikacje danych do dostępu przez konstruktory I settery, co zapewnia kontrolę nad poprawnością danych(hermetyzacja danych)

4. **P:W jaki sposób program zabezpiecza obiekty przed ustawieniem niepoprawnych danych, na przykład ujemnej pensji, liczby godzin lub prowizji?**

O:Zastosowano walidacje danych (np. If(pensja<0)) oraz kontrole wejścia w funkcji main, która sprawdza poprawność strumienia std::cin oraz zakresy wartości przed samym utworzeniem obiektu.

5. **P:Na czym w tym projekcie polega praktyczne zastosowanie hermetyzacji?**

O:Polega ona na odzieleniu interfejsu(metody publiczne) od implementacji I danych(pola prywatne). Użytkownik klasy korzysta z gotowych metod, nie mając względu ani możliwości bezpośredniego zepsucia wewnętrznej struktury danych obiektu.

6. **P:Jakie elementy klasy bazowej zostały odziedziczone przez klasy pochodne?**

O:Odziedziczone zostały wszystkie pola prywatne(dostępne poprzez metody oraz konstruktor bazy) oraz deklarację metod wirtualnych jak obliczPensje() I wyswietl().

7. **P:Jakie zalety dało użycie dziedziczenia w tym projekcie?**

O:Pozwoliło na uniknięcie powtarzania kodu dla pól wspólnych oraz umożliwiło stworzenie polimorficznego kontenera std::vector<Pracownik*>, który może zarządzać różnymi typami pracowników w sposób jednolity

8. **P:Czy ten program dałoby się napisać bez dziedziczenia? Jeśli tak, to czym różniłoby się takie rozwiązanie?**

O:Tak, wtedy byłaby po prostu jedna klasa z polem enum w celu wyboru typu pracownika oraz wymagana byłaby dodatkowa instrukcja switch do każdej metody I niepotrzebne marnowanie pamięci na nieużywane pola dla danego typu umowy. Skalowalność takiego rozwiązania na dłuższą metę byłaby by bardzo niska

9. P:Które metody zostały zadeklarowane jako wirtualne i dlaczego?
O:Metody, które zostały zadeklarowane jako wirtualne to np. Destruktor(dla bezpiecznego usuwania), obliczPensje() oraz wyswietl(). Pozwala to na wywołanie właściwej wersji metody w zależności od typu obiektu wywołanej przez wskaźnik klasy bazowej
10. P:W jaki sposób program wykorzystuje polimorfizm dynamiczny?
O:W pętli przechodzącej przez wektor wskaźników Pracownik* . Program dopiero w czasie wykonywania rozstrzyga, czy pod wskaźnikiem kryje się etatowiec, godzinowiec. Uruchamia odpowiednią logikę obliczeń.
11. P:Co się stanie, jeśli metoda obliczPensje() nie będzie wirtualna?
O:Kompilator zastosuje wczesne wiązanie i wywoła wiązanie z metodą bazową. Ponieważ w tym projekcie jest ona czysto wirtualna, brak słowa virtual(i implementacji) uniemożliwiłoby kompilację oraz wywołanie poprawnej logiki.
12. P:Jakie są największe zalety podejścia obiektowego w tym zadaniu?
O: Łatwa rozszerzalność, dodanie nowego pracownika wymaga dodania dodatkowej klasy, zamiast modyfikacji logiki działania programu czy klasy bazowej
13. P:Jakie są ograniczenia podejścia obiektowego w tym zadaniu?
O: Należy do nich konieczność ręcznego zarządzania pamięcią w przypadku **new** i **delete** oraz minimalny narzut na tablicę metod wirtualnych